织梦 DreamWeave 技术全案

目录

1 项目概述 1

1.1 项目定位 1

1.2 核心亮点 1

2 整体技术架构 1

2.1 技术栈总览 2

2.2 模块与能力对应 2

3 核心模块代码实现 3

3.1 个性化学习画像引擎 3

3.2 AI 辅导教师系统 5

3.3 智能出题与批改系统 10

3.4 沉浸式 Web 交互体验 16

3.5 后端 AI 服务统一封装 20

4 AI-Web 赛道核心创新点 22

4.2 Web 前端创新 22

4.3 工程化落地优势 23

5. 结语 23

6.未来展望 24

1 项目概述

1.1 项目定位

织梦 DreamWeave 是一款面向编程初学者的 AI 个性化编程学习平台。平台以 “学、练、测、评” 一体化为主线，将大语言模型深度融入编程学习全链路，围绕个性化学习画像、苏格拉底式 AI 辅导、智能出题与批改、沉浸式交互体验四大核心能力，帮助用户从零起步、循序渐进地掌握编程技能。项目采用前后端分离架构，前端基于 HTML5、CSS3 与原生 JavaScript，结合 Canvas、WebGL 等可视化技术构建极客风格的交互界面，后端基于 Spring Boot 提供统一的 AI 能力服务，实现了从代码诊断、知识分析、AI 对话、错题训练到成长报告的全流程闭环。

1.2 核心亮点

1. 极客沉浸式学习体验：采用“暗黑星空+代码编辑器”，精准击中程序员的审美偏好，有效提升学习专注度。

2. 全链路数据反馈闭环：打通“代码诊断—知识分析—AI 对话—错题训练—成长报告”的完整数据流，真正实现数据驱动的自适应学习。

3. AI 教学价值的重塑：将 AI 从单纯的“问答机器”转变为“编程学习伙伴”，注重思维引导而非答案输出。

2 整体技术架构

本项目采用前后端分离架构，将大语言模型深度融入编程学习全链路，打造集“学、练、测、评”于一体的个性化 AI 编程学习平台。

2.1 技术栈总览

技术层级	核心技术栈	关键说明
前端表现层	HTML5 + CSS3 + JavaScript + Canvas 2D + WebGL + GLSL + Web Speech API	负责页面结构与交互、动态视觉效果渲染、多模态输入交互
后端业务层	Java + Spring Boot + RESTful API + Jackson	提供业务接口、请求处理、JSON 数据序列化与 AI 能力编排
AI 服务层	DeepSeek 大模型 + Prompt 工程 + 学习画像上下文注入	统一大模型调用，支撑出题、批改与个性化辅导

2.2 模块与能力对应

核心模块	业务能力	技术实现核心
学习画像引擎	学情统计、薄弱点识别、个性化学习支持	buildLearningProfile()、正确率统计算法、基于学习画像的规则推荐
AI 辅导教师	启发式对话、多轮上下文、个性化指导	后端对话组装逻辑、苏格拉底式 System Prompt
智能练习中心	AI 动态出题、多语言多题型支持、智能批改	后端出题 Prompt 构建、前端判分 + AI 代码分析
沉浸式交互系统	极光背景、星空粒子、答对礼花反馈	WebGL GLSL 着色器、Canvas 粒子系统
AI 基础服务	大模型统一调用、参数管理、异常处理	后端 DeepSeekService 封装类

3 核心模块代码实现

3.1 个性化学习画像引擎

学习画像是平台个性化能力的核心底座，基于历史答题数据计算知识点掌握度、识别薄弱项，为 AI 辅导、智能出题提供数据支撑。

3.1.1 学习画像构建核心逻辑

构建用户个性化学习画像，计算各知识点正确率、识别薄弱项、统计学习时长与连续学习天数。

```
function buildLearningProfile() {
    const data = getData();


    // 逐个知识点计算掌握情况
    const stats = topics.map(function (topic) {
        const records = data.history.filter(function (item) {
            return item.topic === topic;
        });
        const correct = records.filter(function (item) {
            return item.correct;
        }).length;
        return {
            name: topic,
            attempts: records.length,
            correct: correct,
            accuracy: records.length
                ? Math.round(correct / records.length * 100)
```

```
            : 0
    };
});
// 提取正确率最低的 3 个知识点作为薄弱项
const weakTopics = stats
    .filter(function (item) {
        return item.attempts > 0;
    })
    .sort(function (a, b) {
        return a.accuracy - b.accuracy;
    })
    .slice(0, 3)
    .map(function (item) {
        return item.name;
    });
// 返回完整画像数据
return {
    total: data.total,
    correct: data.correct,
    accuracy: data.total
        ? Math.round(data.correct / data.total * 100)
        : 0,
    streak: getStreak(data.minutes),
    todayMinutes: data.minutes[today()] || 0,
    topics: stats,
    weakTopics: weakTopics
```

```
    };
}
```

3.1.2 基于学习画像的薄弱知识点分析

以下代码为基于学习画像的推荐思路示意，实际系统通过学习画像识别薄弱知识点，并为 AI 辅导提供参考。

```
// 获取当前用户学习画像
const profile = buildLearningProfile();
// 提取薄弱知识点列表
const weakTopics = profile.weakTopics || [];
// 优先选择正确率最低的知识点作为出题目标
const targetTopic = weakTopics.length > 0
    ? weakTopics[0]
    : null;
// 组织 AI 出题请求参数
const requestData = {
    language: currentLanguage,
    topic: targetTopic
};
```

3.2 AI 辅导教师系统

采用“苏格拉底式”启发教学理念，通过严格的 Prompt 约束确保 AI 只引导不代写，同时动态注入学情数据，实现个性化辅导。

3.2.1 后端：多轮对话与学情注入机制

保留最近 14 轮对话上下文，将用户学习画像拼接到系统提示词中，兼顾对话连贯性与辅导针对性。

```
// 构建对话消息列表
List<Message> history = request.messages() == null
        ? List.of() : request.messages();
List<Map<String, String>> messages = new ArrayList<>();

// 注入带学情画像的系统提示词
messages.add(Map.of(
    "role", "system",
    "content", tutorSystemPrompt(request.profile())
));

// 保留最近 14 轮对话，控制 Token 消耗
int start = Math.max(0, history.size() - 14);
for (int i = start; i < history.size(); i++) {
    Message item = history.get(i);
    if (item == null || item.content() == null) {
        continue;
    }
    String content = item.content().strip();
    if (content.isEmpty()) {
        continue;
    }
    // 单条消息长度截断，防止上下文溢出
    if (content.length() > 4000) {
        content = content.substring(0, 4000);
    }
```

```
    String role = "assistant".equals(item.role())
            ? "assistant" : "user";
    messages.add(Map.of(
        "role", role,
        "content", content
    ));
}
```

3.2.2 苏格拉底式教学系统提示词

严格约束 AI 教学行为，坚持引导式教学，避免直接输出答案。

```
/**
 * 生成 AI 辅导教师系统提示词，注入用户学习画像
 */
private String tutorSystemPrompt(Map<String, Object> profile) {
    return """
            你是「织梦 DreamWeave」的 AI 辅导教师，
            面向编程初学者。

            教学原则：
            1. 全程使用中文，语气耐心、鼓励，不评判学生；
            2. 优先用提问、类比和提示引导学生自己思考，
               不要一上来就给完整答案；
            3. 学生明确要求答案或连续卡住时，
               再给出分步骤讲解；
            4. 代码放在 Markdown 代码块中，
               并解释关键步骤；
```

```
            5. 单次回答尽量控制在 350 字以内，除非需要解释代码。
            6. 严格约束 AI 只引导不代写　优先采用启发式引导，在必要时提供分步骤讲解。

            学生画像：%s
            """.formatted(profileSummary(profile));
}
private String profileSummary(Map<String, Object> profile) {
    int total = number(profile.get("total"));
    int accuracy = number(profile.get("accuracy"));
    int streak = number(profile.get("streak"));
    String weak = joinList(profile.get("weakTopics"));

    StringBuilder text = new StringBuilder();
    text.append("累计完成 ")
        .append(total)
        .append(" 题，正确率 ")
        .append(accuracy)
        .append("%，连续学习 ")
        .append(streak)
        .append(" 天");
    if (!weak.isBlank()) {
        text.append("，薄弱知识点：")
            .append(weak);
    }
    return text.toString();
}
```

3.2.3 前端：多模态对话交互

支持语音与文本双输入，管理对话上下文，实现流畅问答体验。

```
// 提交对话请求
async function sendTutorMessage(text) {
    // 获取并校验用户输入
    const content = String(text || "").trim();
    if (!content) return;

    // 保存用户消息到对话历史
    tutorMessages.push({
        role: "user",
        content: content
    });
    appendTutorMessage("user", content);
    // 调用后端 AI 辅导接口，传入最近 12 轮对话 + 当前学习画像
    const data = await postAI("/api/ai/tutor", {
        messages: tutorMessages.slice(-12),
        profile: buildLearningProfile()
    });
    const reply = data && data.reply;
    // 保存 AI 回复，形成完整对话闭环
    tutorMessages.push({
        role: "assistant",
        content: reply
    });
    appendTutorMessage("assistant", reply);
```

```
}
```

3.3 智能出题与批改系统

系统结合本地题库与 DeepSeek 大模型动态出题机制，支持选择题和编程题两种练习模式，并提供自动判分与 AI 辅助代码分析功能。

3.3.1 后端：AI 动态出题核心

支持题型切换，强制 JSON 格式输出并做校验，保证题目结构规范可用。

```
// 根据题型生成对应 Prompt
String userPrompt = switch (type) {
    case "choice" -> choicePrompt(
        language, topic, scenario, excludedTitles
    );
    case "coding" -> codingPrompt(
        language, topic, scenario, excludedTitles
    );
    default -> null;
};
// 调用大模型生成题目
String content = deepSeek.complete(
    List.of(
        Map.of("role", "system", "content", SYSTEM_PROMPT),
        Map.of("role", "user", "content", userPrompt)
    ),
    true,  // 开启 JSON 模式
    0.95
```

```
);
// 解析并校验题目格式
JsonNode question = parseJson(content);
if ("choice".equals(type)) {
    validateChoice(question); // 选择题格式校验
} else {
    validateCoding(question); // 编程题格式校验
}
```

3.3.2 知识点引导式题目生成原理示意

配套知识点引导式题目生成 Prompt 模板，支持多语言、多知识点定制。

```
String prompt = """
    请根据以下学习要求生成编程练习题：
    编程语言：%s
    目标知识点：%s

    要求：
    1. 题目符合指定知识点
    2. 提供明确的题目描述
    3. 提供答案与解析
    4. 返回 JSON 格式
    """.formatted(language, topic);
```

3.3.3 前端：选择题自动批改与错题记录

实时判分、记录错题、触发动效反馈，同步更新学习画像数据。

```
// 提交答案判分逻辑
document.getElementById("submitAnswerBtn").addEventListener("click", function () {
```

```
if (!currentQuestion || answered) return;
if (selectedAnswer === null) {
    alert("请先选择答案！");
    return;
}
answered = true;
// 判断正误
const correct = selectedAnswer === currentQuestion.answer;
const data = getData();

// 生成题目唯一标识，用于错题记录
const questionKey = currentQuestion.questionKey ||
    (currentQuestion.source === "AI 出题" ? "ai:" : "local:") +
    normalizeQuestionTitle(currentQuestion.title);
currentQuestion.questionKey = questionKey;

// 更新全局答题数据
data.total++;
if (correct) {
    data.correct++;
}

// 记录答题历史，错题保留完整题目信息
const record = {
    topic: currentQuestion.topic,
    correct: correct,
```

```
    date: today(),
    questionKey: questionKey
};
if (!correct) {
    record.question = {
        questionKey: questionKey,
        topic: currentQuestion.topic || "综合",
        language: currentQuestion.language || "",
        title: currentQuestion.title || "",
        options: Array.isArray(currentQuestion.options)
            ? currentQuestion.options.slice()
            : [],
        answer: currentQuestion.answer,
        explain: currentQuestion.explain || "",
        source: currentQuestion.source || "练习题"
    };
}
data.history.push(record);
saveData(data);

// 展示结果与解析
const result = document.getElementById("answerResult");
result.textContent =
    (correct ? "回答正确" : "回答错误") +
    "\n\n 正确答案：" +
    currentQuestion.options[currentQuestion.answer] +
```

```
        "\n\n 解析：" +
        currentQuestion.explain;
    result.hidden = false;

    // 答对触发礼花动效
    if (correct && typeof window.dreamweaveConfetti === "function") {
        window.dreamweaveConfetti();
    }
    // 切换按钮状态，适配普通练习/错题练习模式
    document.getElementById("submitAnswerBtn").hidden = true;
    const nextButton = document.getElementById("nextQuestionBtn");
    if (wrongPracticeMode) {
        nextButton.textContent = wrongPracticeQueue.length <= 1
            ? "再练一次 →"
            : (wrongPracticeIndex + 1 >= wrongPracticeQueue.length
                ? "重新练一轮 →"
                : "下一道错题 →");
    } else {
        nextButton.textContent = aiChoiceMode ? "换一道题 →" : "下一题 →";
    }
    nextButton.hidden = false;
    // 更新首页数据看板
    updateDashboard();
});
```

3.3.4 编程题 AI 智能批改

针对编程主观题，调用大模型从多维度给出评价与改进建议。

```
async function submitCodingAnswer(code) {
    // 构建批改 Prompt
    const prompt =
        "请批改下面这道 " + currentCodingLanguage + " 编程题的学生答案。\n" +
        "题目：" + currentCodingQuestion.title + "\n" +
        (currentCodingQuestion.requirements
            ? "要求：" + currentCodingQuestion.requirements + "\n"
            : "") +
        "\n 学生代码：\n" + code + "\n\n" +
        "请按以下格式回答：\n" +
        "1. 总体评价\n" +
        "2. 存在的问题\n" +
        "3. 改进建议\n" +
        "4. 参考代码";
    // 调用 AI 接口获取批改结果
    const response = await fetch("/api/ai/chat", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({
            question: prompt
        })
    });
```

```
    const data = await response.json();
    const answer = data.choices?.[0]?.message?.content;
    // 渲染批改反馈
    feedback.textContent = answer;
}
```

3.4 沉浸式 Web 交互体验

采用暗黑极客风设计，通过 WebGL、Canvas 实现动态背景与交互反馈，提升学习沉浸感。

3.4.1 WebGL 动态极光背景

基于分形布朗运动实现流动极光效果，支持时间与鼠标交互。

```
// GLSL 分形布朗运动：叠加多层噪声生成自然纹理
float fbm(vec2 p) {
    float value = 0.0;
    float amplitude = 0.5;
    for (int i = 0; i < 4; i++) {
        value += amplitude * noise(p);
        p *= 2.03;
        amplitude *= 0.5;
    }
    return value;
}
// 时间驱动与鼠标位移交互
float t = uTime * 0.05;
vec2 shift = (uMouse - 0.5) * 0.16;
// 生成动态噪声场
```

```
float f = fbm(p * 1.7 + q * 1.35 + vec2(t * 0.3, 0.0));

// 极光渐变配色
vec3 col = deep;
col = mix(col, sky, g1 * 0.55);
col = mix(col, cyan, g2 * 0.45);
col = mix(col, indigo, g3 * 0.35);
```

3.4.2 星空粒子连线效果

Canvas 实现随机粒子运动，近距离粒子自动连线，营造科技感星空背景。

```
/**
 * 创建单个星空粒子
 */
function createParticle() {
    var color = COLORS[Math.floor(Math.random() * COLORS.length)];
    return {
        x: Math.random() * width,
        y: Math.random() * height,
        vx: (Math.random() - 0.5) * 0.24,
        vy: (Math.random() - 0.5) * 0.24,
        r: Math.random() * 1.4 + 0.5,
        color: color,
        alpha: Math.random() * 0.35 + 0.15
    };
}
```

```
// 粒子运动与连线渲染
function renderParticles() {
    // 更新粒子位置
    particles.forEach(p => {
        p.x += p.vx;
        p.y += p.vy;
    });
    // 近距离粒子连线
    for (let i = 0; i < particles.length; i++) {
        for (let j = i + 1; j < particles.length; j++) {
            let a = particles[i];
            let b = particles[j];
            var dx = a.x - b.x;
            var dy = a.y - b.y;
            var distance2 = dx * dx + dy * dy;
            if (distance2 < 14400) {
                var alpha = (1 - Math.sqrt(distance2) / 120) * 0.13;
                ctx.strokeStyle = "rgba(56, 189, 248," + alpha + ")";
                ctx.beginPath();
                ctx.moveTo(a.x, a.y);
                ctx.lineTo(b.x, b.y);
                ctx.stroke();
            }
        }
    }
```

```
}
```

3.4.3 答题正确礼花反馈

答对题目时触发礼花动效，提供正向学习反馈。

```
window.dreamweaveConfetti = function () {
    if (reduceMotion) return;
    var pieces = [];
    // 初始化 90 个礼花碎片
    for (var i = 0; i < 90; i++) {
        pieces.push({
            x: window.innerWidth / 2,
            y: window.innerHeight * 0.34,
            vx: (Math.random() - 0.5) * 13,
            vy: (Math.random() - 0.95) * 13,
            size: 6 + Math.random() * 7,
            rotation: Math.random() * Math.PI * 2,
            spin: (Math.random() - 0.5) * 0.32
        });
    }
    // 逐帧更新粒子物理运动
    var frame = function (now) {
        for (var p = 0; p < pieces.length; p++) {
            var piece = pieces[p];
            piece.vy += 0.22;    // 重力加速度
            piece.vx *= 0.99;    // 空气阻力
            piece.x += piece.vx;
```

```
            piece.y += piece.vy;
            piece.rotation += piece.spin;
        }
        // 渲染与动画循环
        requestAnimationFrame(frame);
    };
    requestAnimationFrame(frame);
};
```

3.5 后端 AI 服务统一封装

对 DeepSeek API 进行统一封装，屏蔽调用细节，为上层业务提供标准化的大模型调用能力。

```
@Service
public class DeepSeekService {

    @Value("${deepseek.api.key}")
    private String apiKey;

    @Value("${deepseek.api.url}")
    private String apiUrl;

    private final ObjectMapper mapper = new ObjectMapper();
    private final HttpClient httpClient = HttpClient.newHttpClient();

    /**
     * 通用大模型对话调用方法
```

```
     * @param messages 对话消息列表
     * @param jsonMode 是否开启 JSON 输出模式
     * @param temperature 生成温度
     * @return 模型生成内容
     */
    public String complete(List<Map<String, String>> messages,
                           boolean jsonMode, double temperature) {

        Map<String, Object> requestBody = new LinkedHashMap<>();
        requestBody.put("model", "deepseek-chat");
        requestBody.put("messages", messages);
        requestBody.put("temperature", temperature);
        requestBody.put("max_tokens", jsonMode ? 3200 : 1600);

        // JSON 模式强制返回结构化数据
        if (jsonMode) {
            requestBody.put("response_format",
                    Map.of("type", "json_object"));
        }

        String json = mapper.writeValueAsString(requestBody);

        // 构建 HTTP 请求
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(apiUrl))
            .timeout(Duration.ofSeconds(120))
```

```
            .header("Content-Type", "application/json")
            .header("Authorization", "Bearer " + apiKey)
            .POST(HttpRequest.BodyPublishers.ofString(
                json, StandardCharsets.UTF_8))
            .build();

        // 发送请求并获取响应
        HttpResponse<String> response = httpClient.send(
            request,
            HttpResponse.BodyHandlers.ofString(
                StandardCharsets.UTF_8)
        );

        // 解析并返回模型回复
        JsonNode root = mapper.readTree(response.body());
        return root.path("choices").get(0).path("message").path("content").asText();
    }
}
```

4 AI-Web 赛道核心创新点

4.1 AI 教育技术创新

1.学情驱动的上下文注入个性化机制：将用户学习画像动态注入 AI 导师提示词，为个性化辅导提供学情依据，解决通用 AI 回答泛化、无针对性的问题。

2.约束型启发式对话算法：通过严谨的 Prompt 工程实现苏格拉底式教学，严格限制 AI 直接输出答案，真正实现 “授人以渔” 的编程教学。

3.大模型动态题库生成：本地题库与 AI 动态出题相结合，支持多语言、多知识点的灵活练习，按需实时生成多语言、多知识点题目，内容可无限扩展。

4.2 Web 前端创新

1. 双引擎沉浸式视觉：极光背景 + 星空粒子的暗黑极客风 UI，贴合程序员审美，提升学习专注度与产品辨识度。

2. 全链路正向反馈动效：通过答题礼花、学习数据动态更新与 AI 多轮对话反馈，将学习结果与交互体验相结合，增强学习过程的参与感。

3. 多模态交互入口：语音 + 文本双输入，降低提问门槛，适配不同学习场景。

4.3 工程化落地优势

1. 前后端分离架构：模块解耦，便于扩展与迭代。

2. 大模型服务统一封装：大模型服务统一封装，可快速替换底座模型。

3. 浏览器本地学习数据管理：采用 LocalStorage 保存答题历史、错题记录及学习统计信息，为学习画像构建和成长报告提供数据支持。

5. 结语

“织梦”平台成功将 AI 技术落地于编程教育场景。以“AI 个性化教学”为核心，将学习画像、启发式引导、动态出题、沉浸式交互与工程化落地融为一体，通过极客美学的 UI 设计、深入的大模型集成、启发式的教学机制以及可视化的数据追踪，构建了一个完整的“学练测评”闭环，实现了从传统刷题到个性化智能学习的范式升级。系统不仅提高了学习效率，更培养了学生独立思考、自主调试代码的能力。并且项目已完整实现前端交互与核心 AI 能力联调，具备良好的可扩展性与演示效果。

6.未来展望

引入代码沙箱 (Code Sandbox) 后续计划接入 Docker 容器化沙箱技术，在前端网页提供隔离、安全的代码运行环境。经过 AI 诊断、纠错、优化后的代码片段，无需用户本地配置开发环境，即可直接在网页端一键执行、查看运行结果与报错日志，真正实现边写边测、即时验证。沙箱会做资源限制、权限隔离与超时销毁，防止恶意代码执行，保障平台安全。同时支持多语言代码运行、标准输入输出、简单文件读写，搭配 AI 自动解析运行报错，把运行异常同步给诊断模块，形成【编写 —AI 诊断 — 在线运行 — 反馈调优】闭环，降低初学者配置环境、调试代码的门槛。

多人在线协同 搭建多人实时协作编码工作台，基于协同编辑协议实现多用户同步编辑、光标位置实时显示、版本增量保存，支持分组学习、结对编程、小组项目开发。协同过程中 AI 同步提供实时辅助：可以针对所有人的公共代码给出统一提示，也可以单独为每位成员推送个性化答疑、代码审查建议；支持权限划分（查看 / 编辑 / 评论）、在线语音 / 文字讨论区、代码批注、协作历史回溯。后续配套协作任务广场、小组作业提交、导师巡看评审能力，打造面向编程学习者、兼具练习、交流、项目共创能力的编程学习社区。

更复杂的 Agent 机制 升级平台 AI Agent 能力，不再局限于单次问答、单段代码纠错。Agent 可深度理解学习者的长期目标、现有基础、可用学习时长、薄弱知识点，自主拆解长周期学习路径，自动生成跨度为数周甚至数月的项目式学习计划。Agent 具备动态感知与自我调整能力：会持续跟踪用户练习数据、诊断报告、成长档案，判断掌握程度；如果某类知识点反复出错，自动放缓进度、插入配套练习、微课、小项目；当用户完成阶段性目标，自动解锁进阶任务与综合实战项目。同时支持目标切换、计划暂停、任务优先级调整，Agent 自动重排学习路线，兼顾知识点递进、实战落地，引导学习者从零散刷题转向体系化、项目驱动式自主学习。